CSE 2421

Lecture 2.1

Brief History of C

- Around for 40 years
- Originally designed by Dennis Ritchie at AT&T Bell Labs
- C now standardized by ISO
- Compilers exist for many processor architectures
- One of the most widely used languages of all time
- The basis for many, many other languages
  - C++, C#, Java, Python, Perl, PHP, and more

C vs Java

- Java
  - Source Code
    ↓
  - Byte Code
    ↓
  - Virtual Machine (JVM)
    ↓
  - Machine Code

```
a=a+1

iload_1
iload_2
iadd
istore_3

(interprets byte code)

(a lot)
```

C vs Java

- C
  - Source Code
    ↓
  - Machine Code

```
a=a+1
```

```
inc eax
```

C

- Low overhead, extremely fast
  - Runs on “bare metal”
- No frills
  - No classes
  - No garbage collection

C

- Portable
- Versatile
- Simple
- Straightforward
- WYSIWYG!
- Requires minimal (no) runtime support

C

- Pros
  - You control everything
  - You have ultimate power

- Cons
  - You control everything
  - You have ultimate power

C vs Java

- You control everything, so you have to *do* everything
  - What takes 1 line in Java might take 100 in C
- You have ultimate power
  - Which makes it easy to shoot yourself in the foot
  - C will let you do (dangerous) things that Java would not
- Similar syntax
  - Blessing, curse
- Much, much, *much* smaller standard library
  - DIY
- Not object oriented

C vs Java

- C
  - Direct memory manipulation
  - Platform/Architecture dependent primitives (eg. ptrs)
  - Manually managed memory
  - All variables declared up front
  - No exception handling
- Java
  - Primitive types plus object references
  - Platform independent primitives
  - Garbage collected memory
  - Just-in-time (JIT) declaration
  - Exception handling

C does not…

- C does not
  - Support the notion of Classes or Objects
  - Support encapsulation
    - Everyone can see everything all the time
  - Support inheritance
  - Support overloading
- C *does*, however, have all the necessary features to let us build such a system

C

- Lacks many modern and useful constructs
  - A reasonable price for the control it gives
- Simple syntax, few keywords, few concepts

A minimal, correct C program

```
int main(void)
{
    return 0;
}
```

Breaking it down

A minimal, correct C program

```
int main(void)
{
    return 0;
}
```

Function Name

int main(void)

- Every full C program begins inside a function called "main()“
  - A function is simply a collection of language statements that do "something"
- The main function is always called when the program first executes
  - From main, we can call other functions

A minimal, correct C program

```
int main(void)
{
    return 0;
}
```

Function Arguments

int main(void)

- Valid ‘main’s
  - `int main() {…}`
  - `int main(void) {…}`
  - `int main(int argc, char* argv[]) {…}`
  - `int main(int argc, char** argv) {…}`

int main(void)

- Compare to Java
  - Java has a main() as well
  - But note that ours is not embedded within a class.
  - C does not have classes.
    - All methods (simply known as functions or procedures) are written at file scope.

int main(void)

- `main(int argc, char* argv[])`
  - `char* argv[]` – array of arguments
  - `int argc` – number of arguments
- Compared to Java
  - The main() method in Java has the prototype ‘main(String[] args)’ which provides the program with an array of strings containing the command-line parameters
  - In C, an array does not know its own length so an extra parameter (argc) is present to indicate the number of entries in the argv array.
    - We’ll discuss this more later

A minimal, correct C program

```
int main(void)
{
    return 0;
}
```

Function's Return Value

A minimal, correct C program

```
int main(void)
{
    return 0;
}
```

Function Body

A minimal, correct C program

```
int main(void)
{
    return 0;
}
```

C Statement

A minimal, correct C program

```
int main(void)
{
    return 0;
}
```

Return value

A minimal, correct C program

```
int main(void)
{
    return 0;
}
```

All statements in C end in a semi-colon

Hello, world!

```
#include <stdio.h>

int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

Compiling

- Compile:

  gcc hello.c -o hello
- Run:

  ./hello

Compiling Demo

Hello, world! dissected - #include

- `#include <stdio.h>`
  - You don’t need to write everything yourself!
  - You can “include” things written by others (or written by you, in another file)
  - The standard C library contains a wealth of useful functions you can use
  - To “get” them into your program, you #include the header containing the declaration of the function you want (more on this later)
  - stdio.h gives us access to the printf function

Hello, world! dissected - main

- `int main(void)`
  - main is called when the program launches
  - This is our definition for main
  - Our definition accepts no parameters
    - void
    - main can accept a list of strings, or nothing
  - Our definition returns an integer
    - int
    - main must return an integer

Hello, world! dissected – {}

- { }
  - Curly braces
  - i.e. "block begin" and "block end“
  - The code in between is called a “block”
  - The current “block” is a function
    - So the code between the curly braces defines the function
  - Blocks can be nested inside each other

Hello, world! dissected - printf

- `printf()`
  - The standard function used to print output
  - Definition is part of the standard C library
  - How can we find out what printf does?
    - Google it
    - Or (more leet)
      - `man 3 printf`

Hello, world! dissected - return

- `return 0`
  - “Functions” vs. “Procedures”
    - Functions return a value
      - `int f()`
    - Procedures do not
      - `void f()`
    - Many people will use the term “function” for both
    - By default, functions in C return an int
      - “f() {…}” is equivalent to “int f() {…}”
      - *DO NOT* use this antiquated construct
      - Explicitly define the return value
      - If there is no return value, use void

Hello, world! dissected - return

- `return 0`, continued
  - If your function is not declared with void
    - It needs to return a value
    - The ‘return’ statement specifies the value to return
  - In C conditionals
    - 0 means false
    - Non-zero means true
  - In Unix/Linux
    - 0 means success

Hello, world! dissected - return

- `return 0`, continued
  - So what does “return 0” mean?
    - In this context, it means the function ran successfully
    - We can make this more clear
    - stdlib.h defines EXIT_SUCCESS as 0
    - This makes the author’s intent much more clear
      - Not “return false”
      - “return success”

Hello, world! dissected - return

- `return 0`, continued
  - C does not have exception handling
  - Errors need to be passed back through return values
  - e.g. man printf
    - printf returns an int
    - The number of characters printed
    - You can check whether or not printf succeeded by looking at its return value
  - The lesson: pass back useful information

Hello, world! dissected - return

- `return` statement
  - Every function, except those returning void, should have at least one
  - *Technically*, it is possible to return from a non-void function without a return statement (that is, to fall through the last })
  - This should not be done, and results in undefined behavior
  - The type of value returned must match the type of the function, or be convertible to that type

Comments

- C89
  - /* block comment */
  - /* block
    comment */
  - Cannot be nested
    - /* I'm a comment /* still a comment */ but not me */
- C99
  - C89 comments
  - // line comment
  - Can (sort of) be nested
    - // I'm a comment // still a comment // and so am I

Choices

- As in any language, there are many ways to do the same thing

```c
#include <stdio.h>
void main(void)
{
    printf("Hello, world!\n");
}
```

```c
#include <stdio.h>
int main(int argc, char **argv)
{
    printf("Hello, world!\n");
    return (0);
}
```

```c
#include <stdio.h>
void main(void)
{
    printf("Hello,");
    printf(" world!");
    printf("\n");
}
```

```c
#include <stdio.h>
int main(int argc, char **argv)
{
    printf("Hello,");
    printf(" world!");
    printf("\n");
    return (0);
}
```

Refactored Hello, world!

```
#include <stdio.h>
#include <stdlib.h>
/*
 * Main Function
 *   Author(s):
 *   Purpose: Controls program, prints Hello, World!
 *   Input: None
 *   Output: Returns Exit Status
 */
int main(int argc, char **argv)
{
     printf("Hello, world!\n");
     return EXIT_SUCCESS;
}
```

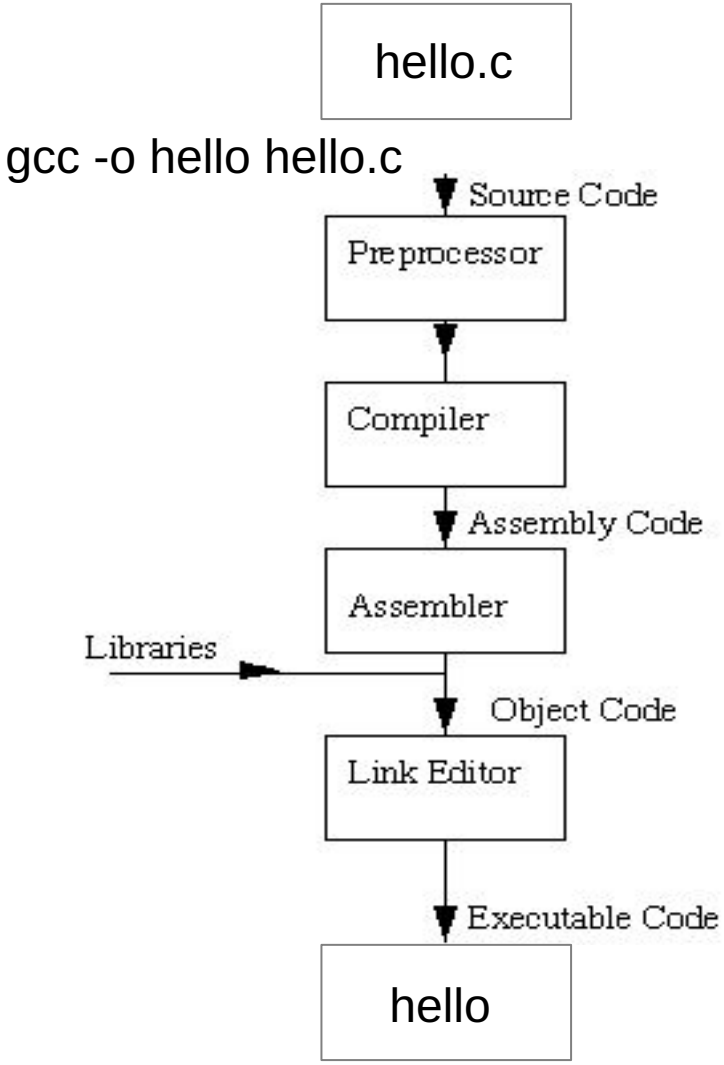
CSE 2421

Libraries, compilers and other basics

Standard C Libraries

- Examples include stdio.h, stdlib.h, float.h, math.h, ctype.h, stddef.h, string.h, errno.h, etc.
- Include a header into your software by instructing the pre-compiler to do so: `#include <stdio.h>`
- Notice, pre-compiler directives do NOT have terminating semicolons
- C is order-dependent, everything must be declared before it's used
- Headers must be included before any declarations, and before any uses of something the header declared

C Compilation Workflow



1) `hello.c` is a plain text file, edit it with any text editor you prefer
2) Invoke `gcc -o hello hello.c`
3) The C *pre-processor* expands all included files, processes all macros: `hello.i`
4) The *compiler* processes the previous output and produces assembly language: `hello.s`
5) The *assembler* assembles this into machine code leaving some unresolved symbols (printf, for example): `hello.o`
6) The *linker* links `hello.o` plus the default runtime libraries into an executable, resolving all symbols (printf): `hello`
7) `hello` can now be executed in the operating system

gcc

- Originally implemented by Richard Stallman
- The gcc compiler is written in C

gcc

- Originally implemented by Richard Stallman
- The gcc compiler is written in C
- **How is this possible?**

Language Overview

- Basic Data Types
- Constants
- Variables
- Identifiers
- Keywords
- Basic I/O

Basic Data Types

- Integer Types
  - Char – smallest addressable unit, typically 8 bits; each byte has its own address
  - Short – not used so much; typically 16 bits
  - Int – default type for an integer constant value; typically 32 bits
  - Long – do you really need it?; counterintuitive, typically 32 bits today
  - Long Long – 64 bits on platforms that support it
- Floating point Types – are “inexact”
  - Float – single precision (about 6 digits of precision)
  - Double – double precision (about 15 digits of precision)
  - constant default unless suffixed with ‘f’

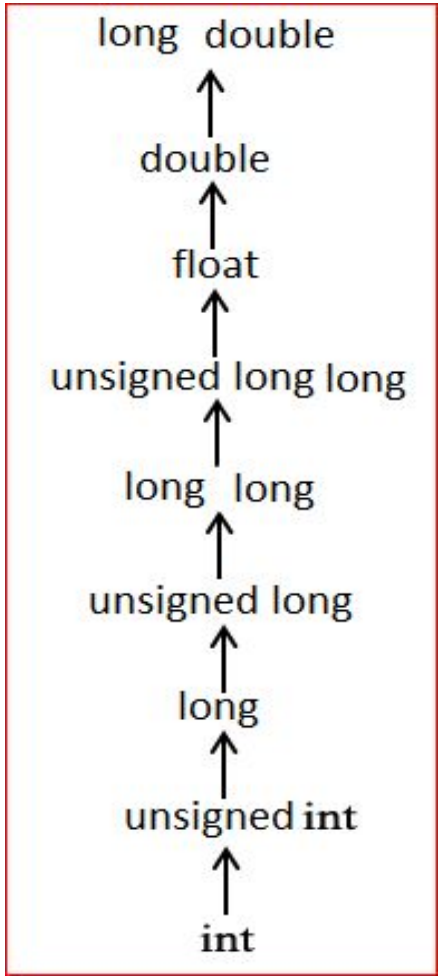
Basic Data Types

- Note that variables of type char are guaranteed to always be one byte.
- There is no maximum size for a type, but the following relationships must hold:

  sizeof(char) <= *sizeof(short)* <= *sizeof(int)*
  <= *sizeof(long)* <= *sizeof(float)*
  <= *sizeof(double)* <= *sizeof(long double)*

C type-casting

The **usual arithmetic conversions** are implicitly performed to cast their values in a common type. Compiler first performs *integer promotion*, if operands still have different types then they are converted to the type that appears highest in the following hierarchy



Arithmetic Expressions - Truncation

Following code is supposed to scale a homework score in the range 0..20 to be in the range 0..100.

```
int score;
// suppose score gets set in the range 0..20 somehow
score = (score / 20) * 100;
```

- Unfortunately, score will almost always be set to 0 for this code because the integer division in the expression (score/20) will be 0 for every value of score less than 20.
- The fix is to force the quotient to be computed as a floating point number...

score = ((double)score / 20) * 100; //OK -- floating point division cast
score = (score / 20.0) * 100; //OK -- floating point division from 20.0
score = (int)(score / 20.0) * 100; //NO -- the (int) truncates the floating quotient back to 0

Derived Types

- Beside the basic types, there is a conceptually infinite class of derived types constructed from the fundamental types in the following ways:
  - arrays of objects of a given type;
  - functions returning objects of a given type;
  - pointers to objects of a given type;
  - structures containing a sequence of objects of various types;
  - unions capable of containing any of one of several objects of various types.
- In general these methods of constructing objects can be applied recursively
  - An array of pointers
  - An array of characters (i.e. a string)
  - Structures that contain pointers

Constants

```
char            'A', 'B'
int             123, -1, 2147483647, 040 (octal), 0xab (hexadecimal)
unsigned int    123u, 2107433648, 040U (octal), 0X02 (hexadecimal)
long            123L, 0x1FFFl (hexadecimal)
unsigned long   123ul, 0777UL (octal)
float           1.23F, 3.14e+0f
double          1.23, 2.718281828
long double     1.23L, 9.99E-9L
```

- Special characters
  - Not convenient to type on a keyboard
  - Use single quotes i.e. ‘\n’
  - Looks like two characters but is really only one

\a	alert (bell) character	\\	backslash
\b	backspace	\?	question mark
\f	formfeed	\’	single quote
\n	newline	\"	double quote
\r	carriage return	\ooo	octal number
\t	horizontal tab	\xhh	hexadecimal number
\v	vertical tab		

Symbolic Constants

- A name that substitutes for a value that cannot be changed
- Can be used to define a:
  - Constant
  - Statement
  - Mathematical expression

Variable Declarations

- Initial value: can be assigned
  - int i, j, k;
  - char a, b, c = ‘D’;
  - int i = 123, j = 456;
  - float f = 3.1415926535;
  - double f = 3.1415926535;

- Type conversion: aka, type casting
  - Be very cautious.
  - (type) identifier;
    - int i = 65;
    - char a; /* range -128 to 127 */
    - a = (char) i; /* What is the value of a? */

Identifier Naming Style

- Rules for identifiers
  - a-z, A-Z, 0-9, and _
  - Case sensitive
  - The first character must be a letter or _
  - Keywords are reserved words, and may not be used as identifiers
- Identifier Naming Style
  - Separate words with '_' or capitalize the first character
  - Use all UPPERCASE for symbolic constant, macro definitions

In-class Exercise

- Determine if the following are valid or invalid variable names. Explain if they are not.

1. goto
2. GoTo
3. main
4. ___
5. ca$h
6. abcdefghijklmnopqrstuvwxyz123456789
7. abcdefghijklmnopqrstuvwxyz123456
8. printf
9. avg value
10. a*b
11. 123
12. buckeye

In-class Exercise - Solutions

- Determine if the following are valid or invalid variable names. Explain if they are not.

1. goto – reserved keyword
2. GoTo
3. main
4. ____
5. ca$h – No special characters
6. abcdefghijklmnopqrstuvwxyz123456789 – yes, But truncated at first 32 characters, making it equivalent to #7
7. abcdefghijklmnopqrstuvwxyz123456
8. printf – conflicts with pre-defined function
9. avg value - no spaces
10. a*b – no special characters
11. 123 – cannot begin with a digit
12. buckeye

Keywords

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

- Purpose: reserves a word or identifier to have a particular meaning
- The meaning of keywords — and, indeed, the meaning of the notion of keyword — differs widely from language to language.
- You shouldn't use them for any other purpose in a C program. They are allowed, of course, within double quotation marks.

Operators

Operator, Category, Duty	Operator, Category, Duty	Operator, Category, Duty
=, Assignment, Equals	!=, Comparison, Is not equal to	>, Bitwise, Shift bits right
+, Mathematical, Addition	&&, Logical, AND	~, Bitwise, One's complement
–, Mathematical, Subtraction	||, Logical, OR	+, Unary, Positive
*, Mathematical, Multiplication	!, Logical, NOT	–, Unary, Negative
/, Mathematical, Division	++, Mathematical, Increment by 1	*, Unary, Pointer
%, Mathematical, Modulo	–, Mathematical, Decrement by 1	&, Unary, Address
>, Comparison, Greater than	&, Bitwise, AND	sizeof, Unary, Returns the size of an object
>=, Comparison, Greater than or equal to	|, Bitwise, Inclusive OR	., Structure, Element access
<, Comparison, Less than	^, Bitwise, Exclusive OR (XOR or EOR)	->, Structure, Pointer element access
<=, Comparison, Less than or equal to	<<, Bitwise, Shift bits left	?:, Conditional , Funky if operator expression
==, Comparison, Is equal to		

Arithmetic Operators

- Mathematical Symbols
  - + - * / %
  - addition, subtraction, multiplication, division, modulus
- Works for both int and float
  - + - * /
- % operator divides two integer operands with an integer result of the remainder
  - 5%2 = 1 (i.e 5/2= 2remainder*1*)
- Precedence – left to right
  - () always first
  - * / %
  - + -

Unary Operators

- ++a and a++ have the behavior of a = a + 1
- --a and a-- have the behavior of a = a – 1
- HOWEVER…
  - ++a → a incremented BEFORE a is used
  - --a → a decremented BEFORE a is used
  - a++ → a is incremented AFTER a has been used
  - a-- → a is decremented AFTER a has been used

```
int main()
{
   int a = 1;
   printf (" a is %d", ++a)
   return 0;
}
/* 2 will be printed  */
```

In both examples the final value of a is 2

```
int main()
{
   int a = 1;
   printf (" a is %d", a++)
   return 0;
 }
/* 1 will be printed */
```

Logical Operators

<<	left-shift
>>	right-shift
&	bitwise AND
|	bitwise OR
^	bitwise exclusive-OR

← wait for these… later

&&	logical AND
||	logical OR

← logical operators

<	less than
>	greater than
<=	less than or equal
>=	greater than or equal
==	equals
!=	does not equal

← relational operators

BITWISE OPERATORS:
0 is FALSE
1 is TRUE

LOGICAL OPERATORS:
0 is FALSE
Everything else is TRUE
Result: 0 if false; 1 if true

Note: C has no keywords as **true** or **false**
`OR', `AND' , and `NOT': differences between bitwise and logical operators.

Logical Operators ex

	a=10 b=0	a=10 b=1	a = 10 b = 3
a & b Bitwise	a=10=1010b b=00=0000b ---------- 0000b 0	a=10=1010b b=01=0001b ---------- 0000b 0	a=10=1010b b=03=0011b ---------- 0010b 1
a && b Logical	a=10=True b=0=False ---------- False	a=10=True b=1=True ---------- True	a=10=True b=3=True ---------- True

For logical operations, Non-Zero values -> True and Zero -> False

Relational Operators

- Used to compare two values
- < <= > >=
- == !=
- Precedence order given above; then left to right
- “else” equivalences (respectively)
  - >= > <= <
  - != ==
- Arithmetic operators have higher precedence than relational operators
- A true evaluates to a nonzero number. A false statement evaluates to zero.
  - For example, the check 0 == 2 evaluates to 0.
  - The check 2 == 2 evaluates to a 1.

Boolean Operators

- C does not have a distinct boolean type
  - int is used instead
- integer 0 = FALSE
- integer non-zero = TRUE

```
i = 0;
while (i - 10) {
   ... }
```

  - while (i-10) > 0 it is considered true
- **Short-Circuit Evaluation**: Relational statements stop evaluating once its value is definitive.
  - In the above example, if x != 0 evaluates to false, evaluation stops regardless of the outcome of y != 0
  - Same for OR if first condition is true
  - We will see why when doing assembly

In class Exercise

```
#include <stdio.h>
main()
{
  int z, a=10, b=0, c=3;
  z = (a > c) ||  (++a > b);
  printf("z = %d  a = %d\n",z,a);

  c = 30;
  z = (a > c) ||  (++a > b);
  printf("z = %d  a = %d\n",z,a);

  z = ! ~ b++ ;
  printf("z = %d  b = %d\n",z,b);

  a = 10, c=3;
  z = ++a * c + a++;
  printf("z = %d  a = %d",z,a);
  return 0;
}
```

In class Exercise

```
{
  int z, a=10, b=0, c=3;
  z = (a > c) ||  (++a > b);

  // work it out
  z = (10 > 3) ||  (++a > b);          // (a > c) -> true (short circuit)
  z = (true) ||  (++a > b);            // right side doesn't get evaluated
  printf("z = %d  a = %d\n",z,a);      // z=1(true)    a=10
}
```

In class Exercise

```
{
  int z, a=10, b=0;
  c = 30;
  z = (a > c) ||  (++a > b);

  // work it out
  z = (10 > 30) ||  (++a > b);
  z = (false) ||  (++10 > 0);
  z = (false) ||  (11 > 0);
  z = (false) ||  (true);

  printf("z = %d  a = %d\n",z,a);                    // z= 1(true)   a=11

}
```

In class Exercise

```
#include <stdio.h>
main()
{

  b=0
  z = ! ~ b++ ;                // hint ! logical and ~ bitwise
  z = ! ~ 0++ ;
  z = ! FFFFFFF++ ;
  z = 0++ ;                    // don't forget to increment b

  printf("z = %d  b = %d\n",z,b);                    // z=0(false)   b=1

 }
```

In class Exercise

```
#include <stdio.h>
main()
{

  a = 10, c=3;
  z = ++a * c + a++;

  // work it out
  z = ++10 * c + a++;
  z = 11 * 3 + a++;
  z = 33 + 11++;
  z = 44++;

  printf("z = %d  a = %d",z,a);                    // z= 44   a=12
  return 0;
}
```

Undefined Behavior in C Specification!

In class Exercise

```
#include <stdio.h>
main()
{
  int z, a=10, b=0, c=3;
  z = (a > c) ||  (++a > b);                    // (a > c) -> true (short circuit
  printf("z = %d  a = %d\n",z,a);               // z=1(true)    a=10

  c = 30;
  z = (a > c) ||  (++a > b);
  printf("z = %d  a = %d\n",z,a);               // z= 1(true)   a=11

  z = ! ~ b++ ;
  printf("z = %d  b = %d\n",z,b);               // z=0(false)   b=1

  a = 10, c=3;
  z = ++a * c + a++;
  printf("z = %d  a = %d",z,a);                 // z= 44   a=12
  return 0;
}
```